

Supplementary Material: The Within-Build Injection Facet of Cargo's Directive Channel

ANONYMOUS AUTHOR(S)

This supplement accompanies the main paper and is not counted toward its page limit. It gives the secondary within-build directive-injection material in full; the main paper keeps only what the cross-unit native-link shadow requires. Section, table, and citation references of the form used in the main text point into that paper.

1 The within-build injection facet

This supplement gives the secondary within-build injection material in full; the main paper's channel section keeps only what the cross-unit shadow needs. None of it is required for the paper's result (a build script already runs arbitrary code, so injecting a directive within its own build is not an escalation), but it establishes that Cargo's directive channel places no trust check on directives, which is the same in-band, unframed, unauthenticated substrate the cross-unit shadow rides.

Injection primitives. Three executed primitives (cargo 1.98.0, oracle a #[cfg(injected)] branch in the built program; the channel-injection driver) exercise the channel's three weaknesses. *Data becomes control* (unframed): interpolating externally sourced data carrying a newline plus cargo::rustc-cfg=injectd makes Cargo apply that second directive, for both the cargo:: and legacy cargo: forms. *A spawned process speaks for the crate* (unauthenticated): a child's directive output is captured and applied, since attribution is positional. *Bounded cross-crate value via DEP_**: a links crate places a chosen single-line value into a direct dependent's DEP_* environment, a chosen value, not an embedded directive, reaching direct dependents only. As a coarse indicator of how common the shape is (not an exploitability rate), a syntactic scan of 472 sampled build scripts finds 113 (23.9%) that both interpolate a computed value into a directive and read it from a non-Cargo source.

The framed, authenticated channel defense. The same abstraction that explains the shadow fixes the injection facet, by a different mechanism family from resolution scoping: authenticate the channel. Framing plus out-of-band delivery moves control off the shared stdout stream (against the three injection scripts, the in-band protocol admits all 3, the framed protocol none), and authorizing the message against a manifest policy Cargo enforces outside the untrusted process rejects 2 undeclared directives while admitting the declared one, closing even a same-process helper, which framing alone cannot. We claim no novelty here: Cackle 0.9 already allowlists emitted instructions per crate. Our measurement only shows such a policy is small to declare, 289 of scripts emit no compilation-affecting directive, median 1 distinct effectful keys. This governs the *emission*; the cross-unit shadow, whose harm is in the *resolution*, is untouched by it and is the paper's subject.

The channel is helper-written in practice. The directive that the shadow rides is routinely not emitted by the package whose build.rs a reviewer reads. Across 121 frozen captures of 14 applications, all 1936 third-party build-script instances link a third-party helper, and 19 helper packages with no build script of their own emit directives from library code (5 compilation-affecting). The same unframed channel also admits a changed helper behind a byte-identical build.rs altering the compiled program (reviewed → attacker-controlled; on an unmodified registry caller, swapping

only autocfg changes what num-traits 0.2.19 compiles), the within-build sibling of the shadow, closed by the framed channel above, not by resolution scoping.

References